

SecureBlog API 프로젝트 보고서

1. 프로젝트 기본 정보

항목	내용
프로젝트명	SecureBlog API
프로젝트 주제	Spring Security와 JWT 인증을 적용한 블로그 백엔드
개발 언어	Java 21
프레임워크	Spring Boot 4.1.0
빌드 도구	Gradle Wrapper
데이터베이스	H2 in-memory DB
인증 방식	Spring Security + JWT Access Token
작성자	1413 손영만
GitHub 주소	https://github.com/sym3872/1413---_Auth---

위 GitHub 주소에 최종 소스 코드와 보고서를 함께 제출한다.

2. 프로젝트 개요

SecureBlog API는 회원가입·로그인을 담당하는 Auth 도메인과 게시글 CRUD를 담당하는 Post 도메인으로 구성된 학습용 백엔드 프로젝트다. 로그인에 성공하면 JWT Access Token을 발급하고, Spring Security 필터가 보호된 API 요청의 JWT를 검증한다.

초보자가 전체 흐름을 이해하기 쉽도록 불필요한 서비스 계층과 인터페이스 분리를 줄였다. DTO는 Controller 내부 record로, Repository는 Entity 내부 인터페이스로, JWT 발급기와 필터는 `SecurityConfig` 내부 클래스로 구성해 주요 Java 파일을 7개로 줄였다. 모든 클래스와 주요 메서드에는 코드의 필요성과 동작을 설명하는 한글 주석을 추가했다.

3. 개발 목표

- Auth와 Blog 두 개의 도메인을 구현한다.
- 회원의 비밀번호를 BCrypt로 암호화해 저장한다.
- 로그인 성공 시 JWT Access Token을 발급한다.
- Spring Security로 공개 API와 로그인 필수 API를 구분한다.
- 로그인한 회원만 게시글을 작성할 수 있게 한다.
- 게시글 작성자 본인만 해당 글을 수정·삭제할 수 있게 한다.
- H2 in-memory DB로 별도의 DB 설치 없이 실행할 수 있게 한다.

- Postman 없이도 기능을 실행할 수 있는 터미널 숫자 메뉴를 제공한다.

4. 사용 기술

기술	사용 목적
Spring Boot	애플리케이션 실행과 REST API 구성
Spring Web MVC	HTTP 요청을 Controller 메서드와 연결
Spring Data JPA	User와 Post를 H2에 저장·조회·수정·삭제
Spring Security	API별 인증 여부 결정과 SecurityContext 관리
JWT 0.13.0	JWT Access Token 생성, 서명 확인, 만료 검증
BCrypt	회원 비밀번호 해시 저장과 로그인 비교
H2 Database	설치 없이 사용하는 메모리 DB
Gradle Wrapper	동일한 Gradle 환경에서 빌드·실행
Java HttpClient	터미널 메뉴에서 REST API 호출

Refresh Token, Redis, OAuth, Lombok은 사용하지 않았다.

5. 프로젝트 구조

```

com.example.springblogapi
├── SpringBlogApiApplication.java # Spring Boot 서버와 터미널 메뉴 시작
├── TerminalMenu.java           # 숫자 메뉴, API 호출, 결과 출력
├── auth
│   ├── User.java              # User 엔티티 + UserRepository
│   └── AuthController.java     # 회원가입·로그인 + 요청·응답 DTO
├── post
│   ├── Post.java              # Post 엔티티 + PostRepository
│   └── PostController.java     # 게시물 CRUD + 작성자 검사
├── config
│   └── SecurityConfig.java     # Security 규칙 + JWT 발급기 + JWT 필터

```

TerminalMenu는 데이터베이스를 직접 조작하지 않고 실제 REST API를 호출한다. 따라서 메뉴로 실행해도 Spring Security와 JWT 검증을 그대로 거친다.

6. 기능 명세서

도메인	기능명	입력	처리 결과	인증 조건
Auth	회원가입	이메일, 비밀번호, 닉네임	BCrypt 암호화 후 User 저장	불필요
Auth	로그인	이메일, 비밀번호	정보가 일치하면 JWT Access Token 발급	불필요
Post	게시글 작성	제목, 내용	로그인 회원을 작성자로 저장	필요
Post	목록 조회	없음	모든 게시글 반환	불필요
Post	상세 조회	게시글 번호	해당 게시글 반환	불필요
Post	게시글 수정	게시글 번호, 새 제목, 새 내용	작성자과 로그인 회원이 같으면 수정	작성자 본인
Post	게시글 삭제	게시글 번호	작성자과 로그인 회원이 같으면 삭제	작성자 본인
Menu	로그아웃	없음	메모리에 저장한 JWT 제거	로그인 상태

7. REST API 명세서

HTTP Method	URL	기능	성공 상태	JWT
POST	/api/auth/signup	회원가입	201 Created	불필요
POST	/api/auth/login	로그인 및 Access Token 발급	200 OK	불필요
POST	/api/posts	게시글 작성	201 Created	필요
GET	/api/posts	게시글 목록 조회	200 OK	불필요
GET	/api/posts/{id}	게시글 상세 조회	200 OK	불필요
PUT	/api/posts/{id}	게시글 수정	200 OK	필요 + 작성자 검사
DELETE	/api/posts/{id}	게시글 삭제	204 No Content	필요 + 작성자 검사

주요 실패 상태는 잘못된 입력 400, 로그인 실패·인증 없음 401, 작성자 불일치 403, 게시글 없음 404, 이메일 중복 409다.

8. 데이터 모델

User

필드	설명
id	자동 증가하는 회원 고유 번호
email	로그인에 사용하는 중복 불가 이메일
password	BCrypt로 암호화된 비밀번호
nickname	게시글에 표시할 작성자 이름

Post

필드	설명
id	자동 증가하는 게시글 고유 번호
title	게시글 제목
content	게시글 본문
author_id	작성자 User를 가리키는 외래키

User와 Post는 1:N 관계다. 회원 한 명은 여러 게시글을 작성할 수 있고, 게시글 하나는 작성자 한 명에게 속한다.

9. Auth System 구조

9.1 회원가입 흐름

- 1 클라이언트가 이메일, 비밀번호, 닉네임을 `/api/auth/signup`으로 보낸다.
- 2 `@Valid`가 이메일 형식, 빈 값, 비밀번호 길이를 확인한다.
- 3 `UserRepository.existsByEmail()`이 중복 이메일을 검사한다.
- 4 `BCryptPasswordEncoder.encode()`가 평문 비밀번호를 해시로 변환한다.
- 5 암호화된 비밀번호만 H2의 users 테이블에 저장한다.

9.2 로그인과 JWT 발급 흐름

- 1 클라이언트가 이메일과 비밀번호를 `/api/auth/login`으로 보낸다.
- 2 `UserRepository.findByEmail()`로 회원을 조회한다.
- 3 `PasswordEncoder.matches()`로 입력 비밀번호와 BCrypt 해시를 비교한다.
- 4 인증에 성공하면 `JwtTokenProvider.createToken()`이 이메일을 subject로 담은 JWT를 만든다.
- 5 JWT에는 발급 시간, 60분 만료 시간, HMAC 서명이 포함된다.
- 6 터미널 메뉴는 받은 JWT를 메모리에 저장하고, 보호 API 요청 시 자동으로 사용한다.

9.3 JWT가 필요한 요청의 인증 흐름

```

클라이언트
  ↓ Authorization: Bearer <Access Token>
JwtAuthenticationFilter
  ↓ JWT 서명·만료 검증
JwtTokenProvider
  ↓ subject에서 이메일 추출
UserRepository
  ↓ 실제 User 조회
SecurityContext
  ↓ 인증된 User를 principal로 저장
Spring Security 권한 검사
  ↓
PostController
  
```

- 1 `JwtAuthenticationFilter`가 모든 요청의 Authorization 헤더를 확인한다.

- 2 `Bearer` 로 시작하면 JWT를 분리한다.
- 3 `JwtTokenProvider.getEmail()`이 서명과 만료 시간을 검증한 뒤 이메일을 꺼낸다.
- 4 이메일로 실제 `User`를 조회한다.
- 5 `UsernamePasswordAuthenticationToken`을 만들어 `SecurityContext`에 저장한다.
- 6 Spring Security가 보호 API의 접근을 허용한다.
- 7 `PostController` 내의 `getCurrentLoginUser()`가 `SecurityContext`의 `User`를 가져와 게시글 작성자로 사용한다.

9.4 Spring Security URL 규칙

주소	규칙
<code>/api/auth/**</code>	모두 허용
<code>GET /api/posts, GET /api/posts/**</code>	모두 허용
<code>/h2-console/**, /error</code>	학습·오류 확인을 위해 허용
위 주소를 제외한 모든 요청	로그인 필요

서버 세션을 생성하지 않는 `STATELESS` 방식을 사용한다. 기본 HTML 로그인 화면과 HTTP Basic은 비활성화했다. REST API가 쿠키 세션을 사용하지 않으므로 CSRF도 비활성화했다.

10. 게시글 인가 구조

인증은 “누구인지 확인”하는 과정이고, 인가는 “그 사용자가 해당 기능을 할 수 있는지 확인”하는 과정이다.

- 작성: JWT로 인증된 `User`를 `Post`의 `author`로 저장한다.
- 조회: 목록과 상세 조회는 로그인 없이 허용한다.
- 수정·삭제: 현재 `User`의 `id`와 `post.author.id`를 비교한다. 두 `id`가 다르면 403 Forbidden으로 차단한다.

11. 실행 방법

VS Code 실행

- 1 ZIP 압축을 풀고 `SecureBlog.code-workspace`를 더블클릭한다.
- 2 `SpringBlogApiApplication.java`를 연다.
- 3 오른쪽 위 `Run Code(▶)` 버튼을 누른다.
- 4 실행 콘솔에 숫자 메뉴가 나오면 기능을 선택한다.

터미널 실행

```
./run.sh
```

별도의 Gradle 설치 는 필요하지 않다. 프로젝트의 Gradle Wrapper가 필요한 Gradle을 사용한다.

12. 테스트 시나리오 및 결과

순서	테스트	예상 결과	확인 결과
1	올바른 정보로 회원가입	회원 저장	정상
2	가입한 계정으로 로그인	JWT 발급·자동 저장	정상
3	로그인 전 게시글 작성 시도	로그인 안내 후 차단	정상
4	로그인 후 게시글 작성	작성자가 로그인 회원으로 저장	정상
5	로그인 없이 목록·상세 조회	조회 성공	정상
6	작성자가 본인 글 수정	제목·내용 변경	정상
7	작성자가 본인 글 삭제	게시글 삭제	정상
8	이메일 형식·빈 값·글 번호 잘못 입력	쉬운 안내 후 재입력	정상
9	목록 결과 표시	HTTP 번호·JSON 대신 한글 메뉴 표시	정상

`./gradlew test`를 통해 프로젝트 빌드가 성공하는 것을 확인했고, 위 핵심 기능은 터미널 메뉴에서 통합 테스트 했다.

13. 실행 시 유의사항

- 프로젝트는 Java 21을 기준으로 작성했다.
- 파일 하나만 열지 말고 `build.gradle`이 있는 프로젝트 폴더 전체를 열어야 한다.
- 서버는 8080 포트를 사용하므로 다른 프로그램이 8080을 사용 중이면 종료해야 한다.
- H2는 메모리 DB이므로 프로그램을 종료하면 회원과 게시글이 사라진다.
- 프로그램을 종료할 때는 가능하면 0번 메뉴를 사용한다.
- JWT는 60분 후 만료되므로 인증이 풀리면 다시 로그인한다.
- 브라우저 주소창은 GET 요청에 적합하다. POST·PUT·DELETE 테스트는 터미널 메뉴, curl 또는 Postman을 사용한다.

자세한 실행 주의사항은 [실행_전_읽어주세요.md](#)에서 확인할 수 있다.

14. 한계점과 개선 방향

- H2 in-memory DB라서 서버가 종료되면 데이터가 유지되지 않는다. 실서비스에서는 MySQL이나 PostgreSQL 같은 영구 DB로 변경할 수 있다.
- Access Token만 사용하므로 토큰을 즉시 폐기하는 서버 기능이 없다. 현재 과제 범위에서는 복잡도를 줄이기 위해 Refresh Token과 Redis를 제외했다.
- JWT 비밀키가 학습용 설정 파일에 있다. 실제 배포 시에는 환경 변수나 Secret Manager로 이동해야 한다.
- 학습을 위해 Controller가 Repository를 직접 사용한다. 규모가 커지면 Service 계층을 추가해 비즈니스 로직을 분리할 수 있다.
- 역할별 권한은 없고 작성자 본인 여부만 검사한다. 향후 ROLE_USER, ROLE_ADMIN 등을 추가할 수 있다.

15. 결론

본 프로젝트를 통해 Spring Boot의 REST API, JPA 엔티티 관계, BCrypt 비밀번호 암호화, JWT 발급·검증, Spring Security의 인증·인가 흐름을 하나의 프로그램으로 구현했다. 특히 SecurityContext에 저장된 현재 회원을 게시글 작성자로 사용하고, 수정·삭제 시 작성자 id를 비교함으로써 Auth 시스템과 CRUD 도메인을 연결했다.